# A Closer Look at Classes

# Contents

# Constructor and Destructor Functions

- C++ allows a constructor function to be included in a class declaration.
- A class's constructor is called each time an object of that class is created.
- Thus any initializations that need to be performed on an object can be done automatically by the constructor function.
- The complement of a constructor is the destructor. This function is called when an object is destroyed.
- The name of a destructor is the name of its class, preceded by a ~.

# Introducing Inheritance

- Inheritance is the mechanism by which one class can inherit the properties of another.
- When one class is inherited by another, the class that is inherited is called the *base class*. The inheriting class is called the *derived class*.

# Object Pointers

- It is possible to access a member of an object via a pointer to that object.
- When a pointer is used, the arrow operator (->) rather than the dot operator is employed.

# In-line Functions

- In C++, it is possible to define functions that are not actually called but, rather are expanded in line, at the point of each call.
- The advantage is that they have no overhead associated with the function call and return mechanism.
- The disadvantage is that if they are too large and called too often, your program grows larger. So, only short functions are declared as inline functions.
- When a short function is defined within a class declaration, it becomes an inline function automatically. The inline keyword is no longer necessary.

# Assigning Objects

- One object can be assigned to another provided that both objects are of the same type.
- When one object is assigned to another, a bitwise copy of all the data members is made.

# Passing Objects To Functions

- Objects can be passed to functions as arguments in just the same way that other types of data are passed.
- Simply declare the function’s parameter as a class type and then use an object of that class as an argument when calling the function.

# Passing Objects To Functions

- When a copy of an object is created because it is used as an argument to a function, the constructor function is not called.
- However, when the copy is destroyed (usually by going out of the scope when the functions returns), the destructor function is called.
- Since, a constructor function is generally used to initialize some aspect of an object, it must not be called when making a copy of an already existing object passed to a function.

# Returning Objects From Functions

- Functions can return objects.
- First declare the function as returning a class type.
- Second, return an object of that type using the normal return statement.

# An Introduction To Friend Functions

- A friend function is not a member of a class but still has access to its private members.
- Inside the class declaration for which it will be a friend, its prototype is included, prefaced by the keyword *friend*.

# Exercises

- Create a Person class with a constructor that initializes the name attribute. Print a message when an object is created.
- Enhance the Person class to have a destructor that prints a message when an object is destroyed. Observe the destructor's behavior.
- Define a base class Shape with attributes like color and a derived class Circle that inherits from Shape. Implement a method in each class to display information.
- Create a BankAccount class with a private balance attribute. Define a friend function that allows external access to modify the balance.
- Enhance the Rectangle class with a method to calculate its area. Write a function that accepts two Rectangle objects and returns the one with the larger area.